Eye-tracking data and BIDS

Pygers-MEG 2026

Eye tracking in the MEG lab

The setup

Setting	MEG lab
Tracker	EyeLink 1000 Plus, OPM mount
Mode	Remote: the camera follows a sticker on the forehead, so no chin rest
Eye	One eye (monocular), 500 samples per second
Calibration	13 points (HV13), then a validation to check the error
Screen	1920 × 1080 px at 120 Hz, 0.94 m from the eye

Remote mode with a sticker

The target sticker gives the tracker a fixed point on the head, so it can follow the eye while the head moves. The sticker's position is saved with every sample: a record of head movement during the run.

Marking events

PsychoPy sends a short text message to the tracker at each event (trial start, stimulus onset, response). The tracker stamps it on its own clock and saves it in the EDF. These messages line the eye data up with the task and the MEG.

What's in an EDF file

One binary file per run, holding two streams of data. SR Research's **edf2asc** turns it into text.

1. Samples

One row every 2 ms (500 Hz)

- Timestamp (ms, on the tracker's clock)
- Eye position x, y (screen pixels)
- Pupil size (area, arbitrary units)
- Target sticker x, y (camera image, 0–10000)
- Sticker distance from the camera (mm) and warning flags

2. Events

Things that happened, with their times

- Eye-movement events from the online parser: fixations, saccades, blinks
- Messages from the tracker: calibration results, recording settings
- Messages your experiment sends: trial starts, stimulus onsets, responses

The online parser

A velocity- and acceleration-based system that turns raw eye position into meaningful events (**saccades**, **fixations** and **blinks**) with low delay, in real time.

It performs well, but has its limits

- Very small saccades may need off-line processing to detect
- Back-up saccades during smooth pursuit may be missed
- It only looks a short time ahead, so post-processing is often needed: merge or discard short fixations, clean up artifacts around blinks

You can tune it

Its settings can be changed, and an existing EDF can be re-parsed with different settings using **edf_reparser**, in the EyeLink Developers Kit.

BIDS for eye-tracking data

Stored as a physiological recording; the **recording-** label (eye1, eye2) is required.

File	What it is	BIDS requirement
…_recording-eye1_physio.tsv.gz	The samples: timestamp, x, y, pupil size (more columns allowed). No header row.	Required
…_recording-eye1_physio.json	Describes the samples. Required fields: SamplingFrequency, StartTime, Columns, PhysioType, RecordedEye, SampleCoordinateSystem	Required
…_recording-eye1_physioevents.tsv.gz	Events and messages: onset, duration, trial_type, message	Optional
…_recording-eye1_physioevents.json	Describes the events columns; requires Columns and OnsetSource	Required with the .tsv.gz
…_events.json	The screen, under StimulusPresentation: ScreenSize, ScreenDistance, ScreenResolution, ScreenOrigin	Required for gaze-on-screen data

Where the files go

- BIDS allows them in any data type folder: meg/, eeg/, func/, beh/, …
- They go with the recording they belong to, with the same sub-, ses-, task- (and run-) labels
- No other recording (behavior only): usually **beh/**
- For MEG: in **meg/**, next to the MEG run

sub-001/ses-001/meg/

```
├── …_task-oddballftcued_meg.fif
├── …_task-oddballftcued_meg.json
├── …_task-oddballftcued_channels.tsv
├── …_task-oddballftcued_events.tsv
├── …_task-oddballftcued_events.json ← both
├── …_recording-eye1_physio.tsv.gz
├── …_recording-eye1_physio.json
├── …_recording-eye1_physioevents.tsv.gz
└── …_recording-eye1_physioevents.json
```

… = sub-001_ses-001_task-oddballftcued

> **Careful with _events.json**
>
> The MEG conversion and eye2bids both write one, with different contents. Merge the two into one file; don't copy one over the other.

eye2bids

URL: https://github.com/mrribbits/eye2bids/tree/scully-config

Converts an EyeLink EDF to BIDS. We use a patched copy of bids-standard/eye2bids, with fixes for the MEG lab's Remote-mode recordings.

What it needs

- The .edf file
- A task name: **--task** (required)
- A metadata file: the MEG lab's comes with the install and is used automatically
- SR Research's edf2asc (Linux: install_edf2asc.sh; Mac: EyeLink Developers Kit)

Install, in your conda environment:

```
pip install
"git+https://github.com/mrribbits/eye2bids.git@<commit>"
```

Run:

```
eye2bids --input_file sub-001_ses-001_task-oddballftcued.edf
\
         --task oddballftcued \
         --output_dir <folder>
```

The Scully MEG metadata file

eyelink_MEG_metadata.yml comes with eye2bids and is used automatically. It holds only BIDS 1.11 fields.

```
# Required
SampleCoordinateSystem: gaze-on-screen
ScreenOrigin: [top, left]
Units: pixel

# Recommended / optional
SoftwareVersion: "5.51.53237"
ScreenRefreshRate: 120
ScreenSize: [0.606, 0.341]
ScreenDistance: 0.94
EyeTrackerDistance: 0.61
RawDataFilters: "EyeLink heuristic file sample filter: EXTRA (level 2)"
InstitutionName: Princeton University, Princeton Neuroscience Institute
InstitutionAddress: 40 Woodlands Way, Princeton, NJ 08540, USA
```

Read from each EDF, not set here

SamplingFrequency, RecordedEye, calibration type, count and errors, PupilFitMethod, EyeTrackingMethod, ScreenResolution, DeviceSerialNumber

TaskName comes from --task

It isn't in the file: give it with **--task** every time (in the notebook: TASK). Use the task- label from your file names, letters and digits only. eye2bids won't run without it.

StartTime and syncing with the MEG

What BIDS asks for

StartTime (required, in _physio.json): seconds from the first MEG sample to the first eye sample. Negative if the eye recording started first.

What eye2bids writes

The first START time on the tracker's own clock, in ms. It isn't relative to the MEG: replace it once the two recordings are synced.

Way to sync	How	Notes
Shared events	PsychoPy sends a tracker message and an MEG trigger at the same moment. Match them and fit an offset and a drift: t_MEG = a + b × t_eye	Most accurate. MNE: mne.preprocessing.realign_raw
One sync event	A single message and trigger at the start of the run	Offset only: clock drift builds up over a long run
Analog output	The EyeLink's analog card sends gaze x, y and pupil to MEG misc (BNC) channels	In sync by design, but no parser events, and the voltages need scaling

Considerations when syncing

Two _events.json files

MNE-BIDS and eye2bids both write one, with different contents.

Merge them; don't let one overwrite the other.

Which sync method?

Shared events is the most accurate:

`mne.preprocessing.realign_raw`

How are the messages written?

For example:

`TRIGGER 1` or `Stimulus Onset`

Each message has to be matched to an event in the MEG _events.tsv, by its value or its name.

Considerations when syncing (continued)

Add the eye data to the FIF as new channels

- Resample the eye data to the MEG rate.
- Fill NaN gaps first: they break resampling and filtering.
- Name the channels so they don't clash with the MEG's analog eye channels.

Don't crop the MEG

- realign_raw crops whichever recording started earlier.
- Pad the eye data at the start and end so only the eye data get cropped.

Where the outputs go

Folder	What goes there, and why
sourcedata/	The original files (.edf, .cMEG): untouched
meg/	Corrected _physio.json (StartTime) and merged _events.json. BIDS raw data is “minimally processed due to file format conversion”, and both files are required to be right. Log the changes in CHANGES.
derivatives/	The synced FIF (resampled, drift-corrected eye data plus the MEG) and the sync report

Harrison's mne-opm pipeline keeps everything in meg/.

Next steps

1. **Explore eye data with notebook 02.**
2. **10/20: Look at how MRI data get converted to BIDS with heudiconv**
3. **10/28: Putting it all together: mne-bids + eye2bids + heudiconv = opm2bids**